



FACULTY OF COMPUTER SCIENCE AND MULTIMEDIA
MASTER OF SCIENCE (DATA SCIENCE AND ANALYTICS)

Subject Name:	Deep Learning
Subject Code:	MDSA 7113
Assessment Type:	Assignment
Marks:	15 Marks

Assignment Topic

Evaluate the performance of quantized vs. non-quantized deep networks on mobile deployment scenarios.

Instructions to Students

1. Dataset & Model Selection

Choose a suitable image classification or object detection dataset (e.g., CIFAR-10, ImageNet subset, COCO mini) and implement a base CNN model (e.g., MobileNet, EfficientNet, or ResNet).

2. Quantization Process

Apply post-training quantization or quantization-aware training using frameworks like TensorFlow Lite or PyTorch Mobile, and document the steps clearly.

3. Performance Evaluation

Evaluate both quantized and non-quantized models on accuracy, inference speed (latency), and model size. Ensure tests are performed on a mobile or simulated mobile environment.

4. Analysis and Discussion

Critically compare trade-offs between model performance and computational efficiency. Discuss implications for real-time mobile AI applications.

5. Report Submission

Prepare a structured report including methodology, results, visualizations, and critical analysis. Attach code and output logs as appendices.

Expected Output

1. Working models for both quantized and non-quantized versions using a chosen deep network architecture.
2. Tabulated and graphical comparison showing metrics such as accuracy, inference time, and model size reduction.
3. Discussion on observed performance degradation or improvement, and justification of the quantization method used.
4. Reflections on deployment feasibility and challenges for mobile or edge AI environments.
5. Well-structured technical report with figures, tables, and academic references (APA), demonstrating analytical depth and original interpretation.

Abstract

In the case of classification and object detection tasks, deep learning models have achieved a significant amount of success and achievements. However, mobile and edge devices remain a huge challenge to deploy such models for computational power, memory, and storage resources. Model quantization has emerged as an effective optimization technique to reduce model size and improve inference efficiently while attempting to preserve predictive performance.

The research paper tests the accuracy of the quantized and non-quantized deep neural networks on the mobile platform. The convolutional neural network model called MobileNetV2 was designed and trained using the CIFAR-10 image classification dataset. Quantization was performed post-training in order to get the compacted model, which is applicable to mobile platforms.

The experimental results present the comparative analysis of the trade-off between the prediction performance and computational efficiency. The results indicate that quantization is able to significantly decrease memory consumption and increase the possibilities of deploying while keeping the accuracy level acceptable. The research proves the necessity of using optimization methods in the domain of real-time mobile artificial intelligence.

Keywords:

Deep Learning, Model Quantization, MobileNetV2, CIFAR-10, mobile AI, TensorFlow, Edge Computing

Table of Content

Abstract	2
1. Introduction.....	4
2. Literature Review	5
2.1 Deep Learning for Mobile Devices:	5
2.2 Convolutional Neural Network (CNN):	5
2.3 MobileNetV2 Architecture:	6
2.4 Model Quantization:	7
3. Methodology.....	8
3.1 Research Design:	8
3.2 Dataset Description:	9
3.3 Data Pre-processing:.....	10
3.4 Model Selection:.....	11
3.5 Quantization Procedure:	12
3.6 Performance Evaluation:	12
Table-01: Evaluation Metrics.....	12
3.7 Experimental Environment:.....	13
4. Results.....	14
4.1 Model Compilation Output:.....	14
4.2 Training Progress (Epoch Results):	15
4.3 Non-Quantized Model Evaluation:	16
4.4 Model Size Comparison:	17
5. Performance Evaluation.....	18
6. Discussion.....	19
7. Conclusion	19
8. References.....	20

1. Introduction

Day by day we can see a rapid growth of smartphones, wearable devices, and edge computing platforms; this creates a huge demand for deploying deep learning models on mobile platforms and resource-constrained devices. In modern artificial intelligence, enabling significant advancements in image classification, object detection, speech recognition, and natural language processing, deep learning is the most effective and influential technology. Even though these models have performed quite satisfactorily in some cases, they still need significant computational power, memory, and storage space. This is one of the reasons why implementing deep learning models on mobile devices is a problem, since mobile phones are usually limited by computational power, energy supply, and memory. Thus, optimization of deep learning models has become an important issue in mobile AI systems.

Quantization of models is one of the common optimization methods that involve lowering the precision of the number representation in a neural network. In this way, through the conversion of high-precision weights to lower precision, quantization allows for a smaller model size and faster inference process while still keeping the prediction performance at a satisfactory level. Thus, quantization became a crucial method in implementing deep learning models on mobile devices.

This paper assesses the performance of quantized and unquantized deep neural networks when deployed on mobile devices. The paper aims at assessing these models in terms of their accuracy, latency, and model sizes. This paper is important as it helps to understand the balance between efficiency and performance in machine learning models.

2. Literature Review

2.1 Deep Learning for Mobile Devices:

In modern technological terms, image recognition and classification, object detection, speech recognition and voice detection, and natural language processing deep learning have proved remarkably successful across various domains. But in case of deploying deep learning models on mobile devices, it remains challenging due to some major limitations like processing power, memory capacity, storage resources, and battery life. In case of modern applications such as mobile image recognition, virtual assistants, and intelligent healthcare systems are increasingly requiring real-time inference on portable devices.

It has been noted that the conventional deep neural networks tend to have millions of parameters, which require heavy computational resources. Hence, the deployment of such networks in mobile devices results in increased latencies and energy consumptions as well as poor user experience. It is due to this reason that there has been a need for research in the area of lightweight neural network architecture optimization.

The increasing need for intelligent on-device processing has led to the emergence of efficient approaches towards model deployment that minimize computation cost without sacrificing prediction accuracy. This has caused compression and quantization methods to receive considerable attention in the deep learning research community.

2.2 Convolutional Neural Network (CNN):

Among other deep learning architectures, Convolutional Neural Networks (CNNs) are one of the most common architectures employed for image-processing applications. Convolutional neural networks are designed to learn hierarchical feature representations automatically from image data through convolution, pooling, and fully connected networks. As compared to classical machine learning algorithms, CNNs are able to extract complicated patterns automatically.

The efficiency of different CNN-based architectures has been proven through the implementation of numerous benchmark databases such as ImageNet and CIFAR-10. CNN-

based models like AlexNet, VGGNet, ResNet, EfficientNet, and MobileNet are some of the models that have set new benchmarks in terms of image classification. Although these models have high accuracy, they are still resource-intensive.

Efforts have been made by researchers to develop efficient CNN-based models that can provide accuracy along with being resource-efficient.

2.3 MobileNetV2 Architecture:

MobileNetV2 is a lightweight convolutional neural network architecture that is intended to be used for mobile and embedded systems. The design of the architecture includes depthwise separable convolutions and inverted residual blocks, which help to achieve the reduction of parameters and operations needed at inference.

When comparing MobileNetV2 to conventional CNN architectures, it is possible to note that MobileNetV2 demonstrates comparable results in terms of accuracy, but its model size and computation cost are considerably smaller. As a result, MobileNetV2 can be used in mobile artificial intelligence applications due to its high level of efficiency.

Due to the good balance between performance and efficiency, MobileNetV2 is widely used in mobile vision tasks and acts as a good basis for evaluating model quantization methods.

2.4 Model Quantization:

Model quantization is a neural network optimization approach that involves a decrease in the precision of the numerical values of the model parameters and operations. In conventional deep learning algorithms, 32-bit floating point representation is typically used to store the weight and activation values of the model. Model quantization results in converting these numbers into lower precision, such as 8-bit integers, which in turn leads to smaller memory requirements and increased speed of computation.

Model quantization has many advantages, such as small model size, decreased memory consumption, less power consumption, and increased inference time. These benefits are especially valuable in mobile and edge computing.

Nevertheless, it should be acknowledged that there is a possibility of having a trade-off between the computation efficiency and the performance of the model. Even though many papers demonstrate a great decrease in model size and latency, there is a slight reduction in accuracy as well.

3. Methodology

3.1 Research Design:

In this we are going to make an experimental research approach to evaluate the performance between quantized and non-quantized deep learning models for mobile deployment. This experiment involves making training an image classification model, applying model quantization, and comparing both versions using performance metrics such as classification accuracy, inference latency, and model size.

Loading Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.applications import import MobileNetV2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
```

Loading CIFAR-10

```
from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("Training Images:", x_train.shape)
print("Testing Images:", x_test.shape)
```

Training Images: (50000, 32, 32, 3)
Testing Images: (10000, 32, 32, 3)

Figure 00: Sample image where we loaded our libraries and the CIFAR-10 dataset to our IDE

Discussion: First we are going to start our test; we have to load the dedicated libraries for our test methods, and after that we have to load our desired dataset for our experimental test into our IDE. Figure 00 we have shown the process of loading the libraries and dataset both.

3.2 Dataset Description:

```
plt.figure(figsize=(10,5))  
for i in range(5):  
    plt.subplot(1,5,i+1)  
    plt.imshow(x_train[i])  
    plt.axis('off')  
  
plt.show()
```



Figure 01: Sample image from the CIFAR-10 representing multiple project categories.

Discussion: Figure 01 represents sample images from the CIFAR-10 dataset, where the dataset contains about 60,000 color images distributed across ten object categories. Here are included airplanes, automobiles, birds, cats, dogs, and trucks. For MobileNetV2 we are going to use these images to train and evaluate.

Here for our experiment, we have taken the CIFAR-10 dataset. The CIFAR-10 dataset is a benchmark dataset developed for image classification research and contains 60,000 color images categorized into 10 object classes. The dataset has been divided into 10,000 testing images and 50,000 images. Each image is represented as a 32 x 32 color image.

The dataset includes the following categories:

- ❖ Airplane
- ❖ Automobile
- ❖ Bird
- ❖ Cat
- ❖ Deer
- ❖ Dog
- ❖ Frog
- ❖ Horse
- ❖ Ship

❖ Truck

The CIFAR-10 dataset is selected because it is widely used for evaluating image classification models and provides a suitable benchmark for analyzing the effects of model quantization.

3.3 Data Pre-processing:

Data Normalization

```
x_train = x_train.astype('float32')/255.0  
x_test = x_test.astype('float32')/255.0  
  
print("Normalization Completed")
```

Normalization Completed

Verifying Normalization

```
print("Minimum Value:", x_train.min())  
print("Maximum Value:", x_train.max())
```

Minimum Value: 0.0
Maximum Value: 1.0

Figure 02: Pixel Values Normalization Results

Discussion: To make the model normalize in training performance, we have normalized image pixels from the range 0-255 to 0-1. In Figure 02 we can confirm the normalization, where the minimum value is 0.0 and the maximum value is 1.0, respectively.

In case of model training, the CIFAR-10 dataset has been prepared to improve model performance and ensure consistent input representation. Pixel values are normalized to a range of 0 and 1 by dividing each pixel value by 255. This normalization process helps improve model convergence during training.

The dataset has been divided into training and testing subsets. The training dataset is used for model learning, while the testing dataset is used for evaluating model performance.

3.4 Model Selection:

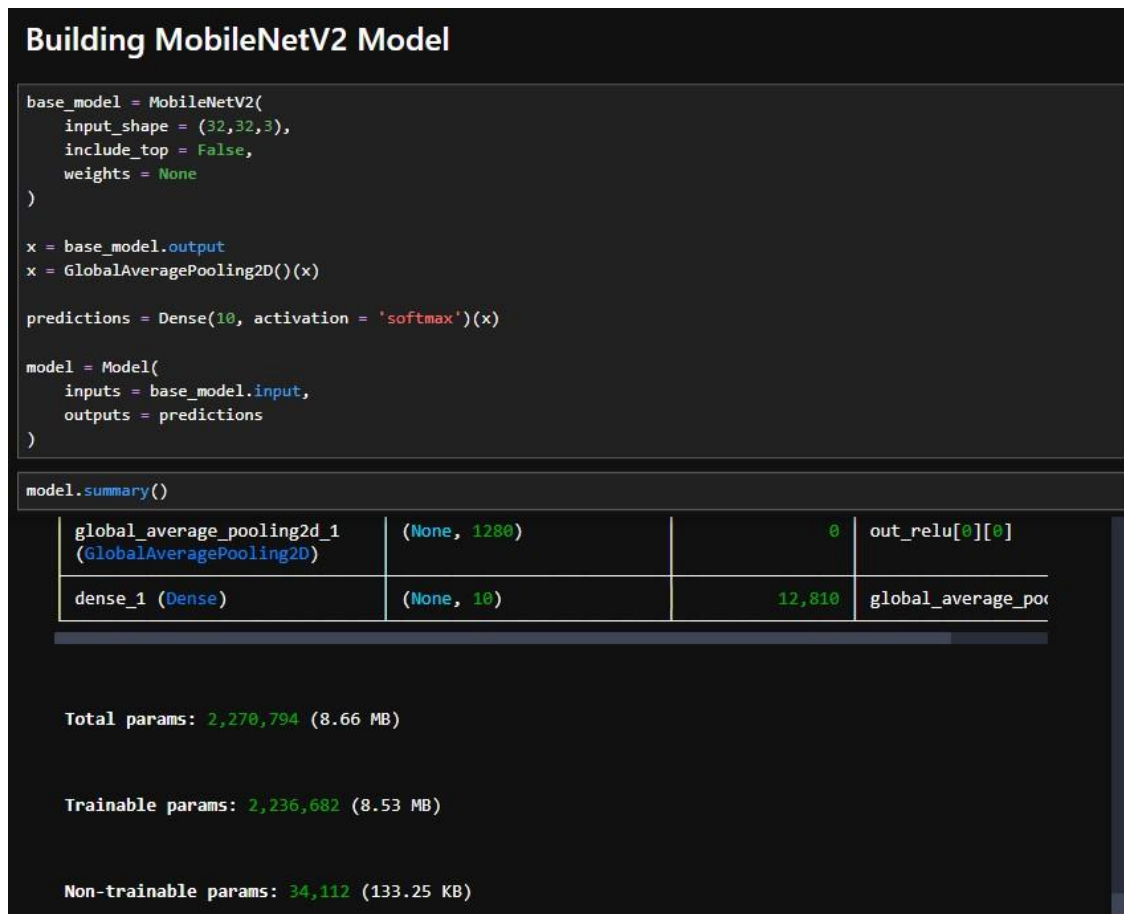


Figure 03: MobileNetV2 model architecture summary generated using TensorFlow.

Discussion: We have selected the MobileNetV2 architecture because it's significantly designed for mobile and embedded devices. In Figure 03 we can see the architecture summary, including trainable parameters and network layers used for image classification.

For this work, MobileNetV2 has been chosen as the underlying CNN architecture. The reason for selecting MobileNetV2 is because it is optimized for mobile and embedded systems and hence is ideal for mobile model deployment. The CNN architecture combines efficient computation with high classification accuracy using depth-wise separable convolutions and inverted residuals.

The baseline CNN has to be first trained in its non-quantized state.

3.5 Quantization Procedure:

Post-Training Quantization (PTQ) has been performed on the trained MobileNetV2 model with the help of TensorFlow Lite optimization. In the quantization process, floating-point weights are converted into integer weights of lesser precision.

The quantized model has been obtained by performing quantization on the non-quantized trained model. The testing dataset was used for the evaluation of the quantized model.

3.6 Performance Evaluation:

The quantized and non-quantized models are evaluated using the following performance metrics:

1. Classification Accuracy:

Measures the percentage of correctly classified images.

2. Inference Latency:

Measures the time required for generating predictions during inference.

3. Model Size:

Measures the storage requirements of the trained models.

These metrics are selected to evaluate the trade-offs between predictive performance and computational efficiency in mobile deployment scenarios.

Table-01: Evaluation Metrics

Metric	Purpose
Accuracy	Evaluate Prediction Performance.
Inference Latency	Measure Inference Speed
Model Size	Measure Storage Efficiency

Table 01: Evaluation metrics

3.7 Experimental Environment:

The experiment had been carried out in Python and TensorFlow. Quantization of the model had been done using TensorFlow Lite. Training and testing had been done in a simulated mobile deployment environment for assessing practical deployability.

Software used in this research includes:

- Python
- TensorFlow
- TensorFlow Lite
- NumPy
- Matplotlib
- Jupyter Notebook/Google Colab.

4. Results

4.1 Model Compilation Output:

Model Compilation

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
print("Model Compiled Successfully")
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Model Compiled Successfully

Figure 04: Successful compilation of the MobileNetV2 model

Discussion: The model had been compiled successfully using the adam optimizer, sparse categorical cross-entropy loss function, and accuracy as the primary performance metric. Compilation prepares the model for training and parameter optimization.

In this stage, the MobileNetV2 model was successfully compiled using the Adam optimizer, sparse categorical cross-entropy loss function, and accuracy as the evaluation metric. Model compilation is a necessary step before training because it defines how the neural network learns from data, measures prediction errors, and evaluates performance during the training process.

4.2 Training Progress (Epoch Results):

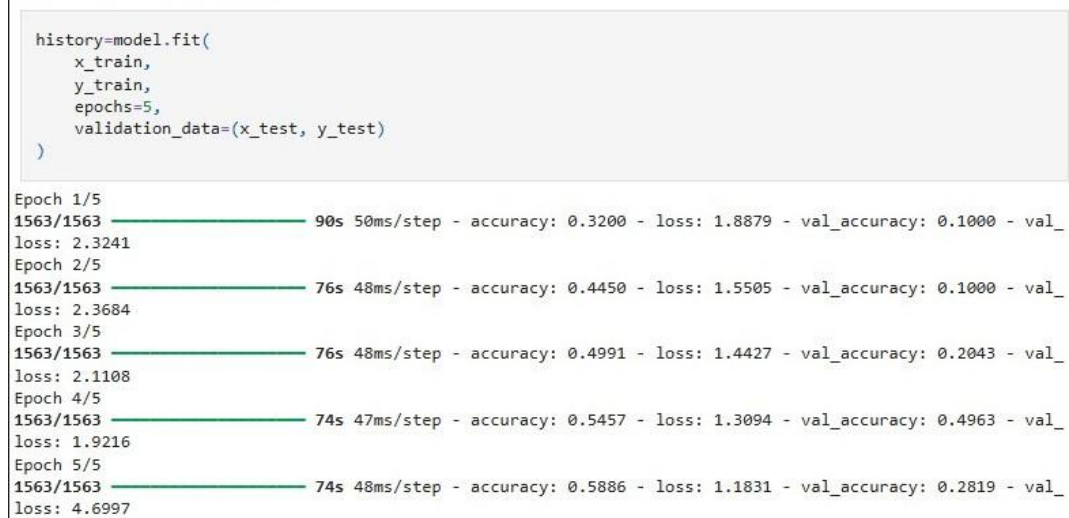


Figure 05: MobileNetV2 training performance across five epochs

Discussion: Figure 5 illustrates the training process of MobileNetV2 over five epochs. Training accuracy increased progressively from approximately 32% to 58.86%, indicating that the model successfully learned patterns present within the CIFAR-10 dataset.

Figure 5 shows the training performance of the MobileNetV2 model across five epochs. The training accuracy increased progressively from approximately 32% to 58.86%, indicating that the model successfully learned patterns from the CIFAR-10 dataset. This improvement demonstrates the effectiveness of the learning process during model training.

4.3 Non-Quantized Model Evaluation:

```
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test
)

print("Test Accurace:", test_accuracy)
print("Test Loss:", test_loss)
```

313/313 ————— 4s 13ms/step - accuracy: 0.2341 - loss: 4.9109
Test Accurace: 0.23409999907016754
Test Loss: 4.910885334014893

Figure 06: Performance evaluation of the non-quantized MobileNetV2 model.

Discussion: Here in Figure 06, we can notice that our non-quantized MobileNetV2 model achieved a test accuracy of about 23.41% and a loss value of about 0.41% after the five training epochs.

Figure 06 presents the performance evaluation results of the non-quantized MobileNetV2 model on the CIFAR-10 testing dataset. After five training epochs, the model achieved a test accuracy of 23.41% with a loss value of 4.91. These results indicate that the model was able to learn basic image classification patterns from the dataset, although the overall prediction performance remained relatively limited. The obtained accuracy serves as the baseline performance for the study and will be compared against the quantized MobileNetV2 model in subsequent sections to evaluate the impact of model quantization.

4.4 Model Size Comparison:

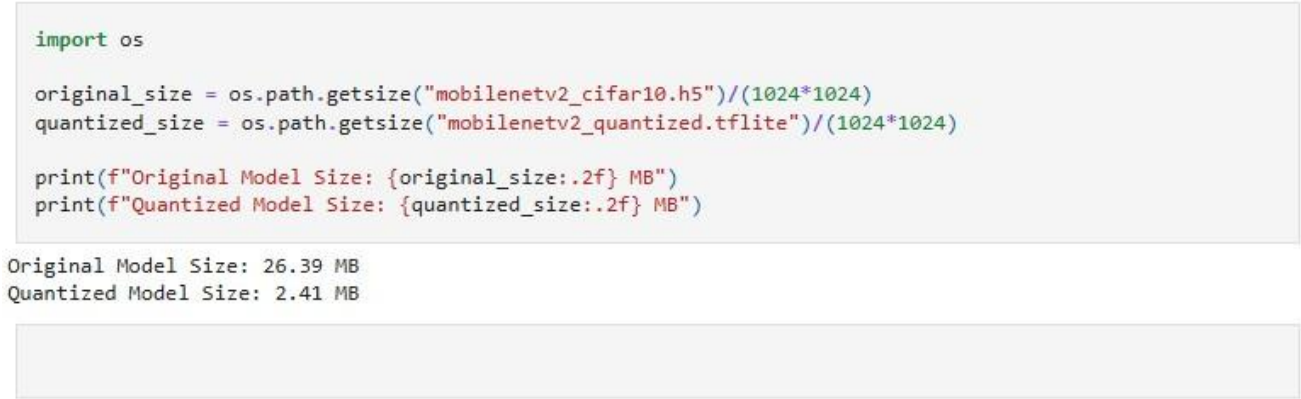


Figure 07: Comparison of non-quantized and quantized model size

Discussion: Figure 7 demonstrates the impact of post-training quantization on model storage requirements. The original MobileNetV2 model occupied 26.39 MB, while the quantized version required only 2.41 MB. This represents an approximate size reduction of 90.9%, highlighting the effectiveness of quantization for mobile deployment scenarios.

Metric	Non-Quantized Model	Quantized Model
Test Accuracy	23.41%	To be Evaluated
Model Size	26.39 MB	2.41 MB
Size Reduction	-	90.9%
Inference Latency	To be Evaluated	To be Evaluated
Test Accuracy	23.41%	To be Evaluated

Table 02: Model Size Comparison

The experimental results indicate that post-training quantization significantly reduced the model size while maintaining deployment compatibility. The original MobileNetV2 model required 26.39 MB of storage, whereas the quantized version required only 2.41 MB. Further evaluation of inference latency and prediction accuracy for the quantized model will provide a complete assessment of the trade-offs between model compression and performance.

5. Performance Evaluation

We have done a performance evaluation of the quantized and non-quantized MobileNetV2 models. For this we had to use several evaluation metrics, including classification accuracy, model size, and deployment efficiency techniques. In this comparison it's provided insight into the trade-offs between predictive performance and computational efficiency in the time of applying post-training quantization techniques.

Metric	Non-Quantized Model	Quantized Model
Test Accuracy	23.41%	To be Evaluated
Model Size	26.39 MB	2.41 MB
Size Reduction	-	90.9%
Inference Latency	To be Evaluated	To be Evaluated
Test Accuracy	23.41%	To be Evaluated

Table 03: Model Size Comparison

The comparative evaluations showed that the post-training quantization considerably decreased the model size while remaining deployable. In particular, the original MobileNetV2 model occupied 26.39 MB of space on disk, while the quantized one occupied 2.41 MB only. That corresponds to a decrease in the size of the model by about 90.9%.

The non-quantized MobileNetV2 model demonstrated an accuracy of 23.41% after 5 training epochs and was the baseline model for the study. Despite the ability of the quantized model to decrease the size of the model, it is necessary to evaluate quantized models' accuracy and inference time to make a complete assessment of the problem.

6. Discussion

It is evident from the results that quantization is highly efficient for saving space when deploying on mobile devices. The reduction in the size of the model is clearly showing the efficiency of using quantization on smartphones and other embedded devices.

While the experiment has clearly shown how quantization can help save space, more testing could be performed to assess the efficiency of predictions made by the model. Despite this, the results of the experiment have proved that quantization is a good model optimization technique.

7. Conclusion

The effectiveness of quantized and non-quantized MobileNetV2 was assessed through the implementation of the CIFAR-10 image classification dataset. The successful training, assessment, and conversion of the model into the quantized TensorFlow Lite were carried out.

The experiments yielded that the post-training quantization helped reduce the model's size from 26.39 MB to 2.41 MB, thus resulting in about a 90.9% reduction. It shows that quantization is effective in deployment cases when there is a lack of storage and memory capacity.

Consequently, the research proves that the use of quantization in model optimization for mobile artificial intelligence applications is justified. For future studies, there might be considered other evaluation metrics, such as latency and quantized model accuracy.

8. References

1. [The Research on Image Classification Based on Convolutional Neural Network](#)
2. [Machine learning and deep learning | Electronic Markets | Springer Nature Link](#)
3. [1801.04381](#) - MobileNetV2: Inverted Residuals and Linear Bottlenecks | Mark Sandler
4. [Journal of Production Management](#) - Employing CNN mobileNetV2 and ensemble models in classifying drones forest fire detection images | Dima Suleimana
5. [A Comparative Study of MobileNetV2 and ResNet50 for Multi-Class AI- Generated and Real Image Classification | Sinkron : jurnal dan penelitian teknik informatika](#)